

HACKATHON IREDE – WEB3

Escrow Freelancer

Protocolo descentralizado de pagamento com garantia on-chain para contratações de freelancers

O PROBLEMA

A falha do mercado tradicional



Taxas Abusivas

Plataformas cobram até 20% sobre o valor do projeto de contratante e freelancer.

até 20%
de taxa



Custódia Centralizada

O dinheiro fica retido em contas bancárias da plataforma, sujeito a bloqueios arbitrários e atrasos nos saques.

7 dias
para sacar



Disputas Opacas

Resoluções de conflito são decididas por equipes humanas em processos opacos, sem mecanismos de transparência ou recurso.

0%
de auditoria



Sem Garantia Real

O freelancer começa a trabalhar sem certeza do pagamento. O contratante paga sem garantia de entrega. A confiança é imposta.

zero
garantia

A SOLUÇÃO

Escrow on-chain. Confiança sem intermediários.

dApp com arquitetura híbrida que elimina o intermediário financeiro, resolvendo o "Problema de Confiança" entre contratantes e freelancers utilizando contratos inteligentes EVM.



Zero Custódia Centralizada

Os fundos (em USDC) são travados em um Smart Contract de Escrow inalterável. Ninguém nem os criadores da Plataforma, pode movimentar esse dinheiro.



Garantia Mútua

O freelancer trabalha sabendo que o dinheiro já está depositado no contrato. O contratante sabe que ele só será liberado mediante entrega do combinado.



Resolução Justa

Em caso de conflito, um árbitro neutro é acionado on-chain para avaliar a situação e direcionar os fundos de forma imparcial.

Por que Web2.5 e não Web3 puro?



O PROBLEMA COM WEB3 PURO

- **Taxa de gás para publicar vaga**

Ninguém quer pagar só para anunciar uma oportunidade.

- **Taxa de gás para se candidatar**

Freelancers abandonam a plataforma antes de começar.

- **Curva de aprendizado Web3**

Carteira, seed phrase, gás... O usuário comum foge.



A SOLUÇÃO: WEB2.5



Experiência fluida no Web2

Cadastro, busca de vagas e candidaturas sem fricção.



Blockchain só onde importa

Escrow, pagamento e auditoria ficam on-chain.



Adoção gradual e natural

O usuário ganha confiança e migra para Web3 sem ser forçado.



"O usuário tem a segurança da blockchain sem precisar saber que ela existe."

Como resolvemos conflitos



OPÇÃO 1

Marketplace Centralizado

A plataforma é o árbitro.

Endereço fixo no contrato.
Freelancer confia na marca.

- ✗ Poder concentrado demais.
- ✗ Contraria a filosofia Web3.

NÃO IMPLEMENTADO



OPÇÃO 2 — MVP

Duplo Consentimento

Contratante propõe árbitro.

Contrato fica em
AGUARDANDO_ACEITE.
Freelancer analisa e decide.

- ✓ Sem concentração de poder.
- ✓ Rápido de implementar.

IMPLEMENTADO



OPÇÃO 3

Kleros / Arbitragem DAO

Jurados anônimos ao redor do mundo.

Teoria dos jogos garante honestidade. Impossível subornar.

- 👤 Mais descentralizado.
- 👤 Alta complexidade técnica.

PRÓXIMA VERSÃO

O que nos diferencia hoje e o que vem a seguir



Factory Pattern

Isolamento total, cada projeto tem seu próprio contrato e cofre



Multitoken

USDC, USDT, DAI, WETH via interface IERC20



Duplo Consentimento

Prestador valida o árbitro proposto antes de assinar o contrato, sem centralização (MVP)



Zero Gas p/ Usuário

Postagem de vagas e candidaturas são gratuitas, gas apenas no momento de criação da escrow

ROADMAP — Evolução do Protocolo

PROD

Árbitro como Plataforma (MVP Atual)

Endereço da equipe age como árbitro. Simples, funcional, mas centralizado, nossa base atual.

✓ Implementado

V2

Duplo Consentimento

Prestador valida e aceita explicitamente o árbitro proposto. Estado ACEITO antes do depósito.

✓ Implementado

V3

Integração Kleros

Corte descentralizada com jurados anônimos e incentivos via teoria dos jogos. ERC-792 arbitrable interface.

→ soon Roadmap

V4

Oráculos de IA (Chainlink Functions)

IA analisa repositório GitHub / assets e emite relatório de conformidade para liberar fundos automaticamente.

→ soon Roadmap

Contratos, Segurança & Integração

🔗 EVENTOS ON-CHAIN (Solidity)

EscrowCriado	<code>Factory.sol</code>	Emissão global após deploy de nova instância via factory.
ContratoAceito	<code>Escrow.sol</code>	Confirmação de aceite e participação do prestador de serviço.
ServicoDepositado	<code>Escrow.sol</code>	Tokens ERC-20 bloqueados com sucesso no contrato inteligente.
ServicoEntregue	<code>Escrow.sol</code>	Sinalização de conclusão; inicia contagem de auto-liberação.
DisputaIniciada	<code>Escrow.sol</code>	Bloqueio total de fundos aguardando resolução via árbitro neutro.
PagamentoRealizado	<code>Escrow.sol</code>	Liberação dos tokens e encerramento do ciclo de vida contratual.

🛡️ SEGURANÇA IMPLEMENTADA

Reentrancy Guard

Estado = FINALIZADO antes de qualquer transfer()
(Checks-Effects-Interactions)

Sem payable desnecessário

Funções de pagamento não são payable, contrato já possui o saldo do token

msg.sender vs tx.origin

Sempre msg.sender para verificar autorização, pois tx.origin é vulnerável a phishing

Isolamento por contrato

Cada projeto tem seu próprio Escrow.sol, se um falha não se propaga aos outros contratos

🔗 INTEGRAÇÃO FRONTEND

•Approve + Execute

- O fluxo exige approve() no token antes do criaEscrow() para garantir a custódia imediata.

•Polling de Estado (Enum)

- Mapeamento dinâmico do estado (0-5) para alternância de botões e visão da UI.

•Automated Triggers

- Scripts podem monitorar block.timestamp para acionar reembolsos ou pagamentos expirados.

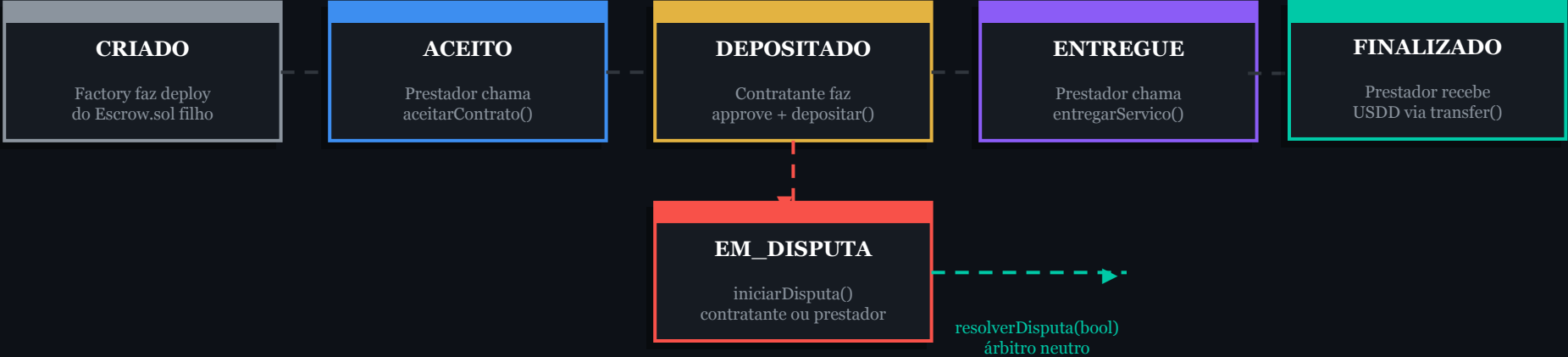
•Logs & Webhooks

- Escuta reativa de eventos via RPC para atualização do banco de dados off-chain.

Estado síncrono Web2+Web3

Após confirmação on-chain, o frontend atualiza o Supabase com o escrow_address e muda o status para em_andamento

Fluxo da Máquina de Estados



FUNÇÕES PRINCIPAIS

criaEscrow()	Factory	Novo Escrow.sol por projeto, permite o isolamento total de fundos	aceitarContrato()	Prestador	Duplo consentimento: prestador valida árbitro proposto	depositar()	Contratante	ERC-20 approve + transferFrom
entregarServico()	Prestador	Salva block.timestamp inicia contagem de 3 dias	pagarPrestador()	Contratante	CEI pattern: estado = FINALIZADO antes do transfer()	pagarPorPrazoExpirado()	Prestador	Auto-release após 3 dias sem resposta do contratante

Obrigado

EQUIPE



Thiago Figueiredo Piazzentin

<https://github.com/Dafloriesz>

<https://www.linkedin.com/in/thiagopiazzentin/>

Confiança não deveria depender de um intermediário.